Assignment 4 (GenAI Comparison)

1. Programs Used

I used ChatGPT and Google Gemini to generate two Sudoku programs in C++. I accessed ChatGPT through my browser and Gemini through its website on the KU computer. I gave both programs the same prompt.

---

2. Prompt Used

“Write a complete C++17 program for EECS 348 Assignment 4 that solves Sudoku puzzles using an object-oriented class and recursive depth-first search with backtracking. Read puzzle1.txt through puzzle5.txt from the current directory. Each file contains a 9x9 Sudoku grid with underscores representing empty cells. For each file, print the filename, original puzzle, and every valid solution, or "No solution found" if none exists. Follow Sudoku row, column, and 3x3 block rules. Include a main function, use standard C++ libraries, and make the code compatible with Linux g++. Provide the complete source code only”

---

3. Comparing the Programs

Both programs use classes and recursive backtracking to solve Sudoku.

Gemini: Uses a Sudoku class and stores the grid as characters. It picks the first empty cell and tries possible numbers. The code is way simpler but doesn't check if the starting puzzle has numbers that conflict with the rules.

ChatGPT: Uses the SudokuSolver class and stores the grid as integers. Then it picks the empty cell with the fewest numbers possible. Then it checks the starting puzzle for any numbers that are invalid. Both Chatgpt’s and Gemini’s programs print the original puzzle and its solutions. Gemini's code is shorter, while ChatGPT's code has better input checking.

---

4. Correctness Tests

I tested the Gemini, ChatGPT, and final programs using the five Sudoku files. All three gave the same results:

puzzle1.txt: 1 solution, puzzle2.txt: 1 solution, puzzle3.txt: 1 solution, puzzle4.txt: No solution, puzzle5.txt: 2 solutions

I tested a completed Sudoku grid with a duplicated number. Gemini's code ran with no problems, but ChatGPT and my final improved program didn't accept it. I also tested a puzzle file with an extra character at the end. Both programs still ran with it, but my final program didn't.

---

5. Execution Time and Space Complexity

I tested each program 70 times in a Linux environment and these were the median times I got:

Gemini: 1.74 ms, ChatGPT: 1.91 ms, Final program: 1.71 ms

The times are very close, so there isn't a big speed difference. Gemini does less checking and ChatGPT uses more time choosing which empty cell to solve first. All of the three programs use backtracking and reccursion, so the worst-case time complexity is about O(9^E), where E is the number of empty cells. In my final program, I used fixed-size arrays instead of vectors for the possible numbers. In order to avoid extra memory allocations while searching.

---

6. Maintainability

Gemini's code is shorter and way easier to follow. ChatGPT's code separates between reading files, checking if numbers are different or the same, solving puzzles, and printing results into different methods. I think that this makes it easier to fix the program or alter it as needed.

7. Program Choice

I chose ChatGPT's program because it checks the starting puzzle first for any errors and picks the empty cell with the fewest possible numbers. I also found its code easier to work with. Both the programs solved the five puzzles. Overall, ChatGPT had better input checking.

---

8. Changes Made to the Program

1)Correctness: I added a check for extra characters in the puzzle files. I also kept the check for invalid starting numbers. If the file has extra characters, the program rejects it.

2)Execution Time: I avoided extra memory allocations while solving by replacing vectors with fixed-size arrays for storing all of the possible numbers.

3)Space Complexity: The program uses a 9x9 array for the board, and as we know, Recursion uses O(E) space, where E is the number of empty cells.

4)Maintainability: Instead of repeating numbers, I added constants and also comments to explain the code.

5)Final Testing: I compiled the final program on a KU Linux computer. There were no errors or warnings. The five puzzles gave 1, 1, 1, 0, 2 as the program's solutions.

Gemini Original Code:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

class Sudoku {
private:
  char grid[9][9];

  bool isValid(int row, int col, char num) const {
    // Check row
    for (int j = 0; j < 9; ++j) {
      if (grid[row][j] == num) {
        return false;
      }
    }

    // Check column
    for (int i = 0; i < 9; ++i) {
      if (grid[i][col] == num) {
        return false;
      }
    }

    // Check 3x3 block
    int startRow = row - (row % 3);
    int startCol = col - (col % 3);
    for (int i = 0; i < 3; ++i) {
      for (int j = 0; j < 3; ++j) {
```

```
            if (grid[i + startRow][j + startCol] == num) {
                return false;
            }
        }
    }

    return true;
}

void solveRecursive(int& solutionCount) {
    int row = -1;
    int col = -1;
    bool isEmpty = false;

    // Find the next empty cell represented by '_'
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (grid[i][j] == '_') {
                row = i;
                col = j;
                isEmpty = true;
                break;
            }
        }
        if (isEmpty) {
            break;
        }
    }

    // If no empty cells remain, a valid solution is found
    if (!isEmpty) {
        solutionCount++;
        std::cout << "Solution " << solutionCount << ":\n";
        printGrid();
        std::cout << "\n";
        return;
    }

    // Try digits '1' through '9'
    for (char num = '1'; num <= '9'; ++num) {
        if (isValid(row, col, num)) {
            grid[row][col] = num;
            solveRecursive(solutionCount);
            grid[row][col] = '_'; // Backtrack
        }
    }
}

public:
Sudoku() {
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            grid[i][j] = '_';
        }
    }
}

bool loadFromFile(const std::string& filename) {
```

```
    std::ifstream file(filename);
    if (!file.is_open()) {
      return false;
    }
    for (int i = 0; i < 9; ++i) {
      for (int j = 0; j < 9; ++j) {
        file >> grid[i][j];
      }
    }
    return true;
  }

  void printGrid() const {
    for (int i = 0; i < 9; ++i) {
      for (int j = 0; j < 9; ++j) {
        std::cout << grid[i][j] << (j == 8 ? "" : " ");
      }
      std::cout << "\n";
    }
  }

  void solve(int& solutionCount) {
    solutionCount = 0;
    solveRecursive(solutionCount);
  }
};

int main() {
  for (int i = 1; i <= 5; ++i) {
    std::string filename = "puzzle" + std::to_string(i) + ".txt";
    Sudoku sudoku;

    std::cout << "Filename: " << filename << "\n";
    if (!sudoku.loadFromFile(filename)) {
      std::cout << "Error: Could not open file " << filename << "\n";
      std::cout << "-----------------------------------------\n\n";
      continue;
    }

    std::cout << "Original Puzzle:\n";
    sudoku.printGrid();
    std::cout << "\n";

    int solutionCount = 0;
    sudoku.solve(solutionCount);

    if (solutionCount == 0) {
      std::cout << "No solution found\n";
    }

    std::cout << "-----------------------------------------\n\n";
  }

  return 0;
}
```

ChatGPT Code

```
#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

class SudokuSolver {
private:
  std::array<std::array<int, 9>, 9> board{};

  bool canPlace(int row, int col, int value) const {
    for (int i = 0; i < 9; ++i) {
      if (board[row][i] == value || board[i][col] == value) {
        return false;
      }
    }

    int boxRow = (row / 3) * 3;
    int boxCol = (col / 3) * 3;
    for (int r = boxRow; r < boxRow + 3; ++r) {
      for (int c = boxCol; c < boxCol + 3; ++c) {
        if (board[r][c] == value) {
          return false;
        }
      }
    }
    return true;
  }

  bool givenCellsAreValid() {
    for (int row = 0; row < 9; ++row) {
      for (int col = 0; col < 9; ++col) {
        int value = board[row][col];
        if (value == 0) {
          continue;
        }
        board[row][col] = 0;
        bool valid = canPlace(row, col, value);
        board[row][col] = value;
        if (!valid) {
          return false;
        }
      }
    }
    return true;
  }

  void search(long long &solutionCount) {
    int selectedRow = -1;
    int selectedCol = -1;
    std::vector<int> selectedCandidates;
    int fewestCandidates = 10;

    for (int row = 0; row < 9; ++row) {
      for (int col = 0; col < 9; ++col) {
```

```
            if (board[row][col] != 0) {
              continue;
            }

            std::vector<int> candidates;
            for (int number = 1; number <= 9; ++number) {
              if (canPlace(row, col, number)) {
                candidates.push_back(number);
              }
            }
            if (candidates.empty()) {
              return;
            }
            if (static_cast<int>(candidates.size()) < fewestCandidates) {
              fewestCandidates = static_cast<int>(candidates.size());
              selectedRow = row;
              selectedCol = col;
              selectedCandidates = candidates;
            }
          }
        }

        if (selectedRow == -1) {
          ++solutionCount;
          std::cout << "Solution " << solutionCount << ":\n";
          printBoard();
          std::cout << '\n';
          return;
        }

        for (int number : selectedCandidates) {
          board[selectedRow][selectedCol] = number;
          search(solutionCount);
          board[selectedRow][selectedCol] = 0;
        }
      }

    public:
      bool load(const std::string &filename) {
        std::ifstream input(filename);
        if (!input) {
          return false;
        }

        for (int row = 0; row < 9; ++row) {
          for (int col = 0; col < 9; ++col) {
            char symbol;
            if (!(input >> symbol) ||
              (symbol != '_' && (symbol < '1' || symbol > '9'))) {
              return false;
            }
            board[row][col] = (symbol == '_') ? 0 : symbol - '0';
          }
        }
        return true;
      }

      void printBoard() const {
```

```
    for (int row = 0; row < 9; ++row) {
      for (int col = 0; col < 9; ++col) {
        if (col > 0) {
          std::cout << ' ';
        }
        if (board[row][col] == 0) {
          std::cout << '_';
        } else {
          std::cout << board[row][col];
        }
      }
      std::cout << '\n';
    }
  }

  long long solveAll() {
    if (!givenCellsAreValid()) {
      return 0;
    }
    long long solutionCount = 0;
    search(solutionCount);
    return solutionCount;
  }
};

int main() {
  for (int puzzleNumber = 1; puzzleNumber <= 5; ++puzzleNumber) {
    std::string filename = "puzzle" + std::to_string(puzzleNumber) + ".txt";
    SudokuSolver solver;
    std::cout << "Filename: " << filename << '\n';

    if (!solver.load(filename)) {
      std::cout << "Unable to read puzzle file.\n\n";
      continue;
    }

    std::cout << "Original Puzzle:\n";
    solver.printBoard();
    std::cout << '\n';

    if (solver.solveAll() == 0) {
      std::cout << "No solution found\n";
    }
    std::cout << "\n";
  }
  return 0;
}
```